Extension 3 — "Ask About Me" Chatbot + Evals

A plain-English explanation of what was built, why, and exactly how it works.

---

1. Why This Task Exists

Calling an AI model is easy — anyone can do it in a few lines of code. The hard, valuable skill is knowing whether the AI is actually answering correctly, and catching the moment a change makes it worse. AI never gives the exact same answer twice, so "I tried it and it seemed fine" proves nothing. This task has two halves on purpose: build the chatbot (the easy half), then build an automatic checklist that proves it works (the real point of the task).

2. What Was Actually Built

A chat box on the portfolio site ("Ask About Me") where a visitor can click ready-made questions, organised into categories (Education, Skills, Projects, Certifications, Other), or type their own question. The bot answers using Ashfaq's real résumé content — nothing invented. Alongside it, a 20-question automatic test suite checks the bot's answers are accurate, safe, and well-behaved, and can prove it catches mistakes if something breaks later.

3. Every File Created — What It Does

`src/lib/resume-context.ts`

> The bot's entire memory of Ashfaq — skills, projects, education, certifications, achievements, all typed out as plain text. This is the ONLY thing the bot is allowed to know about Ashfaq. If a fact isn't written here, the bot should not say it.

`src/lib/chat-system-prompt.ts`

> The rulebook, kept separate from the facts above. Tells the AI things like: only use the facts you're given, never invent a job he didn't have, decline salary questions politely, keep answers to 2-3 sentences, and — if a question is unrelated to Ashfaq — reply with one exact, fixed sentence: "Information not found..." every single time, instead of making up a different excuse each time.

`src/pages/api/chat.ts`

> The actual backend route. When a question comes in, this file combines the rulebook + the facts + the question, sends all of it to Cloudflare's AI model, and sends the answer back — either streamed live (for the real chat box) or as one complete block of text (for the automatic test suite, which is easier to grade that way).

`src/pages/ask.astro`

> The actual webpage a visitor sees. Shows category buttons (Education, Skills, Projects, Certifications, Other) — clicking one shows specific questions, clicking a question gets an instant answer, no typing

required. Also has a free-text box for anything not covered, and a button to download the real résumé PDF.

`src/components/Nav.astro`

The site's navigation bar — updated to add an "Ask About Me" link so visitors actually discover this page exists.

`public/ashfaq-ur-rahman-resume.pdf`

The real résumé file itself, placed where the website can serve it directly as a download.

`evals/cases.ts`

The 20-question checklist. Each entry is one question plus a rule for what a correct answer looks like — for example, "must mention 7.97" or "must NOT mention a dollar amount."

`evals/run-evals.ts`

The automatic grader. Asks the chatbot all 20 questions, checks each answer against its rule, and writes a report showing exactly how many passed, how many failed, and why.

`evals/run-evals.test.ts`

A much smaller, instant check that just makes sure the 20 questions themselves are written correctly (have a question, have a rule) — this one is safe to run on every single code change since it never actually calls the AI.

`.github/workflows/evals.yml`

Tells GitHub to automatically run the full 20-question checklist every time a pull request is opened, and show the results — without blocking the merge, since AI calls take time and cost a little.

`EVALS.md`

The written explanation of the checklist — what each test checks, a sample report, and proof that the checklist actually catches mistakes when something is deliberately broken.

`HLD.md / ARCHITECTURE.md / DECISIONS.md (additions)`

Extra sections added to the project's existing design documents, explaining this specific feature the same way those files already explain everything else.

4. How It Actually Works, Step By Step

Step 1 — A visitor asks something

They click a ready-made question, or type their own, on the Ask About Me page.

Step 2 — The question travels to the backend

The page sends the question to chat.ts on the server.

Step 3 — The backend builds one big message

chat.ts glues together three things: the rulebook (chat-system-prompt.ts), the résumé facts (resume-context.ts), and the visitor's actual question. This combined message is what actually gets sent

to the AI — the résumé facts are repeated on every single question, so the AI always has them fresh, rather than "remembering" them from before.

Step 4 — The AI model reads everything and answers

Cloudflare's AI model (currently GLM-4.7-flash) receives that combined message. It reads through the résumé facts, finds anything relevant to the question, and writes a short answer — following the rules it was given (short, accurate, third-person, no invented facts).

Step 5 — The answer streams back

Rather than making the visitor wait in silence for the whole answer, the model sends it back piece by piece, and the page displays it word by word as it arrives — this is why answers appear to "type themselves out" rather than showing up all at once.

Step 6 — If the question is outside the résumé

The rulebook specifically tells the AI: if the question has nothing to do with Ashfaq, or asks for something the résumé doesn't cover, reply with one exact sentence — "Information not found. Try one of the suggested questions above, or ask something else about Ashfaq." — word for word, every time. This makes the bot's refusal predictable and testable, instead of a different made-up excuse on every attempt.

5. How Accurate Is It, and How Fast?

Accuracy: the bot is only as accurate as the résumé content it's given, plus how well it follows the rules — and this is exactly what the 20-question checklist measures. Each run produces a pass rate (e.g. "18/20 passed"). It isn't claimed to be 100% reliable on every possible phrasing a visitor might type — EVALS.md is explicit about this, listing known blind spots such as no testing against deliberately tricky/adversarial questions, and no testing for occasional inconsistency (each question is only asked once per test run, not repeated to check for flukes).

Speed: answers stream in, so the very first word typically appears in under a second to a couple of seconds, with the full answer finishing a few seconds after that — fast enough that it feels conversational rather than like a slow form submission. Exact speed depends on Cloudflare's model load at that moment, which can vary.

This isn't a guess — it's something you can verify yourself any time by running `npm run evals`, which prints a live pass/fail count and timing for every one of the 20 questions.

6. Why Built This Way (Key Choices)

Cloudflare Workers AI instead of paid APIs: free, already part of the existing tech stack, no extra account or secret key to manage.

All résumé facts sent fresh every time, instead of a fancier "search" system: the résumé is short enough that there's no real benefit to building a more complex retrieval system — that complexity only pays off for much larger documents.

The checklist doesn't block pull requests: AI calls take a few seconds and aren't always perfectly consistent, so making it a hard pass/fail gate could occasionally block unrelated code changes for the wrong reason. It still runs and reports every time, just doesn't lock the door.